

DSA 8070 R Session 2: Matrix Algebra

Whitney Huang, Clemson University

Contents

Overview	2
Data and Summaries	2
Motor Trend Car Road Tests Data	2
From Data Frame to Data Matrix	3
Mean Vector and Covariance Matrix	5
Transformations and Direction	8
Inverse Matrix	8
Orthogonal Matrix Example	9
Eigenvalues and Eigenvectors	9
PCA Preview: Dominant Directions	10
Spectral Decomposition	11
Volume and Dimensionality	12
Determinant, Trace, and Rank	12
Correlation and Generalized Variance	13
Positive Definite Means Positive Variance	13
Quadratic Forms and Distance	14
Euclidean and Mahalanobis Distance	14
Ellipsoid Axes Come from Eigenvectors	15
Standardize and Reconstruct Covariance	15
Square-Root Matrices	15
Partitioning and Synthesis	17
Partitioning Random vectors	17

Overview

Matrix algebra is the mathematical language of multivariate analysis. When each observational unit is measured on several variables, it is natural to organize the data as a matrix and to use matrix operations to describe location, variation, dependence, direction, distance, and dimension.

In this R session, we use the `mtcars` data to connect the matrix formulas from the lecture to direct computation in R. The main workflow is

data matrix $X \rightarrow$ mean vector and covariance $(\bar{\mathbf{x}}, \mathbf{S}) \rightarrow$ directions and transformations $\rightarrow$ volume, dimension, and distance.

The emphasis is not only on obtaining numerical answers, but also on understanding what each matrix quantity means in multivariate analysis.

Data and Summaries

Motor Trend Car Road Tests Data

The built-in `mtcars` dataset contains measurements on 32 automobiles. We select five quantitative variables:

`mpg`: miles per gallon,

`disp`: engine displacement,

`hp`: gross horsepower,

`drat`: rear axle ratio,

`wt`: weight.

Each automobile is one observation and each selected measurement is one variable. Thus, after conversion to a matrix, the rows of X represent observations and the columns represent variables.

```
# Load the built-in Motor Trend automobile data
data(mtcars)

# Select five quantitative variables for the matrix-algebra examples
vars <- which(names(mtcars) %in% c("mpg", "disp", "hp", "drat", "wt"))
cars <- mtcars[, vars]
```

A quick look at the dimensions makes the multivariate data structure explicit.

```
# Number of observations and variables
dim(cars)
```

```
## [1] 32  5
```

```
# Display the first few observations
head(cars)
```

```
##           mpg disp  hp drat   wt
## Mazda RX4    21.0  160 110 3.90 2.620
## Mazda RX4 Wag 21.0  160 110 3.90 2.875
## Datsun 710    22.8  108  93 3.85 2.320
## Hornet 4 Drive 21.4  258 110 3.08 3.215
## Hornet Sportabout 18.7  360 175 3.15 3.440
## Valiant      18.1  225 105 2.76 3.460
```

From Data Frame to Data Matrix

For n observations measured on p variables, the data matrix has the form

$$\mathbf{X} = \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1p} \\ x_{21} & x_{22} & \cdots & x_{2p} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n1} & x_{n2} & \cdots & x_{np} \end{bmatrix}_{n \times p}.$$

Each **row** represents one observation, while each **column** contains all observations for one variable.

```
# Convert the selected variables to a numeric matrix
X <- as.matrix(cars)

# Check the matrix dimensions: n observations x p variables
dim(X)
```

```
## [1] 32  5
```

```
# First observation: one row of X
X[1, ]
```

```
##      mpg    disp      hp   drat     wt
## 21.00 160.00 110.00   3.90   2.62
```

```
# First variable: one column of X
X[, 1]
```

```
##      Mazda RX4      Mazda RX4 Wag      Datsun 710      Hornet 4 Drive
##      21.0           21.0           22.8           21.4
## Hornet Sportabout      Valiant      Duster 360      Merc 240D
##      18.7           18.1           14.3           24.4
##      Merc 230      Merc 280      Merc 280C      Merc 450SE
##      22.8           19.2           17.8           16.4
##      Merc 450SL      Merc 450SLC  Cadillac Fleetwood Lincoln Continental
##      17.3           15.2           10.4           10.4
## Chrysler Imperial      Fiat 128      Honda Civic      Toyota Corolla
##      14.7           32.4           30.4           33.9
##      Toyota Corona      Dodge Challenger      AMC Javelin      Camaro Z28
##      21.5           15.5           15.2           13.3
## Pontiac Firebird      Fiat X1-9      Porsche 914-2      Lotus Europa
##      19.2           27.3           26.0           30.4
## Ford Pantera L      Ferrari Dino      Maserati Bora      Volvo 142E
##      15.8           19.7           15.0           21.4
```

```
# Transpose X: p x n
t(X)
```

```
##      Mazda RX4 Mazda RX4 Wag Datsun 710 Hornet 4 Drive Hornet Sportabout
## mpg      21.00      21.000      22.80      21.400      18.70
## disp    160.00     160.000     108.00     258.000     360.00
```

```

## hp      110.00      110.000      93.00      110.000      175.00
## drat     3.90       3.900       3.85       3.080       3.15
## wt       2.62       2.875       2.32       3.215       3.44
##      Valiant Duster 360 Merc 240D Merc 230 Merc 280 Merc 280C Merc 450SE
## mpg      18.10      14.30      24.40      22.80      19.20      17.80      16.40
## disp     225.00     360.00     146.70     140.80     167.60     167.60     275.80
## hp      105.00     245.00     62.00     95.00     123.00     123.00     180.00
## drat     2.76      3.21      3.69      3.92      3.92      3.92      3.07
## wt       3.46      3.57      3.19      3.15      3.44      3.44      4.07
##      Merc 450SL Merc 450SLC Cadillac Fleetwood Lincoln Continental
## mpg      17.30      15.20      10.40      10.400
## disp     275.80     275.80      472.00     460.000
## hp      180.00     180.00     205.00     215.000
## drat     3.07      3.07      2.93      3.000
## wt       3.73      3.78      5.25      5.424
##      Chrysler Imperial Fiat 128 Honda Civic Toyota Corolla Toyota Corona
## mpg      14.700     32.40      30.400     33.900     21.500
## disp     440.000     78.70      75.700     71.100     120.100
## hp      230.000     66.00      52.000     65.000     97.000
## drat     3.230      4.08      4.930      4.220      3.700
## wt       5.345      2.20      1.615      1.835      2.465
##      Dodge Challenger AMC Javelin Camaro Z28 Pontiac Firebird Fiat X1-9
## mpg      15.50      15.200     13.30      19.200     27.300
## disp     318.00     304.000     350.00     400.000     79.000
## hp      150.00     150.000     245.00     175.000     66.000
## drat     2.76      3.150      3.73      3.080     4.080
## wt       3.52      3.435      3.84      3.845     1.935
##      Porsche 914-2 Lotus Europa Ford Pantera L Ferrari Dino Maserati Bora
## mpg      26.00      30.400     15.80      19.70      15.00
## disp     120.30     95.100     351.00     145.00     301.00
## hp      91.00      113.000     264.00     175.00     335.00
## drat     4.43      3.770      4.22      3.62      3.54
## wt       2.14      1.513      3.17      2.77      3.57
##      Volvo 142E
## mpg      21.40
## disp     121.00
## hp      109.00
## drat     4.11
## wt       2.78

```

Matrix dimensions matter before multiplication. If

$$\mathbf{X} \in \mathbb{R}^{n \times p},$$

then

$$\mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}, \quad \mathbf{X} \mathbf{X}^T \in \mathbb{R}^{n \times n}.$$

The first product summarizes relationships among **variables**, whereas the second summarizes relationships among **observations**.

```

# Cross-product among variables: p x p
crossprod(X)

```

```

##      mpg      disp      hp      drat      wt

```

```
## mpg    14042.310  128705.08   84362.70  2380.2770  1909.7528
## disp  128705.080 2179627.47 1291364.40 25094.7960 27091.4888
## hp     84362.700 1291364.40  834278.00 16372.2800 16471.7440
## drat   2380.277   25094.80   16372.28   422.7907   358.7190
## wt      1909.753   27091.49   16471.74   358.7190   360.9011
```

```
# Cross-product among observations: n x n
# X %*% t(X) # Produces a large n x n matrix
```

Mean Vector and Covariance Matrix

The **mean vector** contains one sample mean for each variable:

$$\bar{\mathbf{x}} = \frac{1}{n} \mathbf{X}^T \mathbf{1} = \begin{bmatrix} \bar{x}_1 \\ \vdots \\ \bar{x}_p \end{bmatrix}.$$

The **sample covariance matrix** summarizes both variation within each variable and covariation between pairs of variables:

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \left(\mathbf{I} - \frac{1}{n} \mathbf{1} \mathbf{1}^T \right) \mathbf{X} = \frac{1}{n-1} (\mathbf{X} - \mathbf{1} \bar{\mathbf{x}}^T)^T (\mathbf{X} - \mathbf{1} \bar{\mathbf{x}}^T).$$

The matrix formula is simply a compact way to compute all p means and all $p \times p$ covariances simultaneously.

```
# Compute the sample mean of each variable using apply()
(mean <- apply(cars, 2, mean))
```

```
##          mpg          disp          hp          drat          wt
## 20.090625 230.721875 146.687500  3.596563  3.217250
```

```
# Store the sample size n and number of variables p
n <- dim(cars)[1]; p <- dim(cars)[2]
```

```
# Convert the data frame to a numeric matrix
X <- as.matrix(cars)
```

```
# Construct the n-vector of ones used in the matrix formulas
ones <- rep(1, n)
```

```
# Compute the mean vector directly from (1/n) X^T 1
(meanCal <- (1 / n) * t(X) %*% ones)
```

```
##          [,1]
## mpg    20.090625
## disp  230.721875
## hp    146.687500
## drat   3.596563
## wt     3.217250
```

```
# Compute the sample covariance matrix using R's cov()
(S <- cov(cars))
```

```
##           mpg          disp          hp          drat          wt
## mpg      36.324103 -633.09721 -320.73206   2.1950635 -5.1166847
## disp -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## hp      -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## drat      2.195064  -47.06402  -16.45111   0.2858814  -0.3727207
## wt       -5.116685  107.68420   44.19266  -0.3727207   0.9573790
```

```
# Reproduce S using the centering-matrix formula
```

```
(Scal <- (1 / (n - 1))) * t(X) %*% (diag(n) - (1 / n) * ones %*% t(ones)) %*% X)
```

```
##           mpg          disp          hp          drat          wt
## mpg      36.324103 -633.09721 -320.73206   2.1950635 -5.1166847
## disp -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## hp      -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## drat      2.195064  -47.06402  -16.45111   0.2858814  -0.3727207
## wt       -5.116685  107.68420   44.19266  -0.3727207   0.9573790
```

```
# Reproduce S by explicitly centering each row of X
```

```
(Scal <- (1 / (n - 1))) * t(X - ones %*% t(meanCal)) %*% (X - ones %*% t(meanCal)))
```

```
##           mpg          disp          hp          drat          wt
## mpg      36.324103 -633.09721 -320.73206   2.1950635 -5.1166847
## disp -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## hp      -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## drat      2.195064  -47.06402  -16.45111   0.2858814  -0.3727207
## wt       -5.116685  107.68420   44.19266  -0.3727207   0.9573790
```

For

$$\mathbf{S} = (s_{jk}),$$

the diagonal and off-diagonal entries have different meanings:

$$s_{jj} = \text{Var}(X_j), \quad s_{jk} = \text{Cov}(X_j, X_k), \quad j \neq k.$$

The covariance matrix is symmetric, so

$$s_{jk} = s_{kj}.$$

Its **diagonal entries** therefore describe marginal variation, while the **off-diagonal entries** describe how pairs of variables vary together.

```
# Diagonal entries are the sample variances
```

```
diag(S)
```

```
##           mpg          disp          hp          drat          wt
## 3.632410e+01 1.536080e+04 4.700867e+03 2.858814e-01 9.573790e-01
```

```
# S is symmetric: S - S^T should be numerically zero
round(S - t(S), 10)
```

```
##      mpg disp hp drat wt
## mpg      0    0  0    0  0
## disp      0    0  0    0  0
## hp         0    0  0    0  0
## drat       0    0  0    0  0
## wt         0    0  0    0  0
```

```
# Convert covariance to correlation to remove units and compare dependence
cor(cars)
```

```
##      mpg      disp      hp      drat      wt
## mpg  1.0000000 -0.8475514 -0.7761684  0.6811719 -0.8676594
## disp -0.8475514  1.0000000  0.7909486 -0.7102139  0.8879799
## hp   -0.7761684  0.7909486  1.0000000 -0.4487591  0.6587479
## drat  0.6811719 -0.7102139 -0.4487591  1.0000000 -0.7124406
## wt   -0.8676594  0.8879799  0.6587479 -0.7124406  1.0000000
```

A central interpretation of the covariance matrix comes from a linear combination of variables. For any vector of weights $\mathbf{a}$, define

$$Y = \mathbf{a}^T \mathbf{X}.$$

Then

$$\text{Var}(Y) = \text{Var}(\mathbf{a}^T \mathbf{X}) = \mathbf{a}^T \Sigma \mathbf{a}.$$

Thus, the quadratic form

$$\mathbf{a}^T \Sigma \mathbf{a}$$

is not merely symbolic: it is the variance of a real linear combination, or equivalently, the variation of the data after projection onto the direction $\mathbf{a}$.

```
# Choose weights for a linear combination of the five variables
a <- c(1, 1, 0, 0, 1)
```

```
# Compute the linear combination for every automobile
Y <- X %*% a
```

```
# Direct sample variance of the projected data
var(Y)
```

```
##      [,1]
## [1,] 14337.02
```

```
# The same variance computed as a^T S a
as.numeric(t(a) %*% S %*% a)
```

```
## [1] 14337.02
```

Transformations and Direction

Inverse Matrix

The identity matrix leaves a vector unchanged:

$$\mathbf{I}\mathbf{x} = \mathbf{x}.$$

An inverse matrix plays the role of “undoing” a linear transformation:

$$\mathbf{x} \xrightarrow{\mathbf{A}} \mathbf{y} \xrightarrow{\mathbf{A}^{-1}} \mathbf{x}, \quad \mathbf{A}^{-1}\mathbf{A} = \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}.$$

For covariance matrices, the inverse appears later in **Mahalanobis distance**, **multivariate normal likelihoods**, and **discriminant analysis**.

```
# Compute the inverse of the sample covariance matrix
S_inv <- solve(S)

# Check that S^{-1} S is approximately the identity matrix
(S_inv %*% S)
```

```
##           mpg           disp           hp           drat           wt
## mpg    1.000000e+00 -6.712237e-14 -3.425998e-14  3.039963e-16 -5.763283e-16
## disp    8.465125e-17  1.000000e+00 -1.228264e-15  6.314000e-18 -1.950304e-17
## hp      3.496914e-17  4.144814e-17  1.000000e+00  1.061775e-18 -3.317849e-18
## drat   -7.709220e-15  1.013106e-13  5.672211e-14  1.000000e+00  5.891006e-16
## wt     -4.844043e-15  1.174255e-13  4.141715e-14 -5.652220e-16  1.000000e+00
```

In numerical work, an inverse can be theoretically defined but unstable to compute if the matrix is nearly singular. The condition number provides a useful warning.

```
# Check numerical conditioning; a very large value suggests instability
kappa(S)
```

```
## [1] 149654.2
```

```
# Verify the inverse more cleanly after rounding numerical error
round(S %*% S_inv, 8)
```

```
##      mpg disp hp drat wt
## mpg    1    0  0    0  0
## disp    0    1  0    0  0
## hp      0    0  1    0  0
## drat    0    0  0    1  0
## wt      0    0  0    0  1
```

If two variables were exact linear copies of one another, the covariance matrix would contain redundant information and would be singular, so a unique inverse would not exist.

Orthogonal Matrix Example

A square matrix $\mathbf{Q}$ is **orthogonal** when

$$\mathbf{Q}^\top \mathbf{Q} = \mathbf{I},$$

which implies

$$\mathbf{Q}^{-1} = \mathbf{Q}^\top.$$

Orthogonal transformations preserve **lengths** and **angles**. Geometrically, they behave like rotations or reflections rather than stretching the data.

```
# Construct a 3 x 3 orthogonal matrix
Q <- matrix(c(2, 1, 2, -2, 2, 1, 1, 2, -2), ncol = 3) / 3

# Check Q Q^T = I
(Q %*% t(Q))
```

```
##           [,1]      [,2]      [,3]
## [1,] 1.000000e+00 1.541976e-17 1.233581e-17
## [2,] 1.541976e-17 1.000000e+00 2.467162e-17
## [3,] 1.233581e-17 2.467162e-17 1.000000e+00
```

We can also verify directly that an orthogonal transformation preserves Euclidean length.

```
# Choose a vector in R^3
x <- c(1, 2, 3)

# Compare the length before and after applying Q
sqrt(sum(x^2))
```

```
## [1] 3.741657
```

```
sqrt(sum((Q %*% x)^2))
```

```
## [1] 3.741657
```

This geometry matters later because PCA rotates the coordinate system toward directions that are naturally aligned with the covariance structure.

Eigenvalues and Eigenvectors

An **eigenvector** is a direction that does not turn under a linear transformation:

$$\mathbf{A}\mathbf{x} = \lambda\mathbf{x}.$$

The vector $\mathbf{x}$ keeps the same direction, while the **eigenvalue** λ tells us how strongly that direction is stretched or shrunk.

For a covariance matrix $\mathbf{S}$, the eigenvectors are especially important because they identify directions of **multivariate variation**.

```
# Compute eigenvalues and eigenvectors of S
eigen <- eigen(S)
```

```
# Verify  $S \mathbf{e}_1 = \lambda_1 \mathbf{e}_1$  componentwise
(S %% eigen$vectors[, 1] / eigen$vectors[, 1])
```

```
##           [,1]
## mpg  18636.79
## disp  18636.79
## hp    18636.79
## drat  18636.79
## wt    18636.79
```

```
# First eigenvalue
eigen$values[1]
```

```
## [1] 18636.79
```

```
# Eigenvectors returned for a symmetric covariance matrix have unit length
t(eigen$vectors[, 1]) %% eigen$vectors[, 1]
```

```
##           [,1]
## [1,]      1
```

PCA Preview: Dominant Directions

For the covariance matrix,

$$S \mathbf{e}_j = \lambda_j \mathbf{e}_j.$$

The interpretation is

$$\mathbf{e}_j = \text{principal direction}, \quad \lambda_j = \text{variance along that direction}.$$

Ordering the eigenvalues as

$$\lambda_1 \geq \lambda_2 \geq \dots \geq \lambda_p$$

means that $\mathbf{e}_1$ is the direction containing the **largest variance**.

The proportion of variance explained by the first k directions is

$$\text{PVE}_k = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^p \lambda_j}.$$

This is the basic geometric idea behind **principal component analysis (PCA)**; PCA will be developed in detail later in the course.

```
# Eigenvalues are the variances along the principal directions
eigen$values
```

```
## [1] 1.863679e+04 1.452987e+03 9.207420e+00 1.368769e-01 1.157957e-01
```

```
# Proportion of total variance explained by each principal direction
eigen$values / sum(eigen$values)
```

```
## [1] 9.272387e-01 7.229067e-02 4.580980e-04 6.810054e-06 5.761202e-06
```

```
# Cumulative proportion of variance explained
cumsum(eigen$values) / sum(eigen$values)
```

```
## [1] 0.9272387 0.9995293 0.9999874 0.9999942 1.0000000
```

Spectral Decomposition

A symmetric matrix can be reconstructed from its eigenvalues and orthonormal eigenvectors:

$$\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^T = \lambda_1 \mathbf{e}_1 \mathbf{e}_1^T + \lambda_2 \mathbf{e}_2 \mathbf{e}_2^T + \cdots + \lambda_p \mathbf{e}_p \mathbf{e}_p^T.$$

Geometrically, a symmetric matrix transformation can be understood as

rotate into eigenvector coordinates $\rightarrow$ rescale along each eigen-direction $\rightarrow$ rotate back.

```
# Create an array to store the five rank-one spectral components
temp <- array(dim = c(5, 5, 5))
```

```
# Construct lambda_i e_i e_i^T for each eigenpair
for (i in 1:5){
  temp[i,,] <- eigen$values[i] * eigen$vectors[, i] %*% t(eigen$vectors[, i])
}
```

```
# Add the five spectral components to reconstruct S
(out <- apply(temp, 2:3, sum))
```

```
##           [,1]      [,2]      [,3]      [,4]      [,5]
## [1,]  36.324103 -633.09721 -320.73206   2.1950635 -5.1166847
## [2,] -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## [3,] -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## [4,]   2.195064  -47.06402  -16.45111   0.2858814 -0.3727207
## [5,]  -5.116685  107.68420   44.19266  -0.3727207   0.9573790
```

```
# Compare the reconstructed matrix with the original covariance matrix
S
```

```
##           mpg      disp      hp      drat      wt
## mpg      36.324103 -633.09721 -320.73206   2.1950635 -5.1166847
## disp -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## hp    -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## drat   2.195064  -47.06402  -16.45111   0.2858814 -0.3727207
## wt    -5.116685  107.68420   44.19266  -0.3727207   0.9573790
```

Volume and Dimensionality

Determinant, Trace, and Rank

Trace, determinant, and rank summarize different aspects of a multivariate matrix:

$$\text{tr}(\mathbf{S}) = \sum_{j=1}^p \lambda_j$$

measures **total marginal variation**,

$$|\mathbf{S}| = \prod_{j=1}^p \lambda_j$$

measures **generalized variance** or multivariate volume, and

$$\text{rank}(\mathbf{S}) = \# \{j : \lambda_j \neq 0\}$$

describes the **effective linear dimensionality**.

Two covariance matrices can therefore have the same total variation but distribute that variation very differently across dimensions.

```
# Trace: total marginal variation  
(trace <- sum(diag(S)))
```

```
## [1] 20099.23
```

```
# The trace is also the sum of the eigenvalues  
sum(eigen$values)
```

```
## [1] 20099.23
```

```
# Determinant: generalized variance / multivariate volume  
det(S)
```

```
## [1] 3951786
```

```
# The determinant is also the product of the eigenvalues  
prod(eigen$values)
```

```
## [1] 3951786
```

```
# Numerical rank: number of linearly independent directions  
qr(S)$rank
```

```
## [1] 5
```

Correlation and Generalized Variance

For two variables with covariance matrix

$$\Sigma = \begin{bmatrix} \sigma_1^2 & \rho\sigma_1\sigma_2 \\ \rho\sigma_1\sigma_2 & \sigma_2^2 \end{bmatrix},$$

the determinant is

$$|\Sigma| = \sigma_1^2\sigma_2^2(1 - \rho^2).$$

As $|\rho|$ approaches 1, the data cloud becomes increasingly concentrated around a line, so its two-dimensional volume shrinks.

At $|\rho| = 1$, the determinant is zero: one direction is perfectly redundant, and the covariance matrix cannot be inverted.

```
# Compare generalized variance for weak and strong correlation
Sigma_weak <- matrix(c(1, 0.2, 0.2, 1), ncol = 2)
Sigma_strong <- matrix(c(1, 0.95, 0.95, 1), ncol = 2)

# Stronger correlation produces a smaller determinant
det(Sigma_weak)
```

```
## [1] 0.96
```

```
det(Sigma_strong)
```

```
## [1] 0.0975
```

Positive Definite Means Positive Variance

For a symmetric matrix $\mathbf{A}$, the expression

$$\mathbf{x}^\top \mathbf{A} \mathbf{x}$$

is a **quadratic form**.

A matrix is **nonnegative definite** if

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} \geq 0$$

for every $\mathbf{x}$, and **positive definite** if

$$\mathbf{x}^\top \mathbf{A} \mathbf{x} > 0$$

for every nonzero $\mathbf{x}$.

For a covariance matrix,

$$\mathbf{a}^\top \Sigma \mathbf{a} = \text{Var}(\mathbf{a}^\top \mathbf{X}),$$

so nonnegative definiteness follows directly from the fact that variance cannot be negative.

Positive definiteness means there is no exact linear redundancy among the variables.

Quadratic Forms and Distance

Euclidean and Mahalanobis Distance

The squared **Euclidean distance** from a point $\mathbf{x}$ to a center $\boldsymbol{\mu}$ is

$$d_E^2(\mathbf{x}, \boldsymbol{\mu}) = (\mathbf{x} - \boldsymbol{\mu})^\top (\mathbf{x} - \boldsymbol{\mu}).$$

Euclidean distance treats one unit as equally meaningful in every variable and ignores correlation. In multivariate data, this can be problematic because variables may have very different scales and may move together.

Mahalanobis distance adapts the distance measure to the covariance structure:

$$d_M^2(\mathbf{x}, \boldsymbol{\mu}) = (\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1} (\mathbf{x} - \boldsymbol{\mu}).$$

It standardizes for different scales and accounts for correlation. Equal Mahalanobis-distance contours are ellipses or ellipsoids aligned with the covariance structure.

```
# Sample center and covariance matrix
center <- colMeans(X)
S <- cov(X)

# Squared Mahalanobis distance of every automobile from the sample center
d2 <- mahalanobis(X, center = center, cov = S)

# Display the first few squared distances
head(d2)
```

```
##           Mazda RX4      Mazda RX4 Wag      Datsun 710      Hornet 4 Drive
##           1.704568        1.141007        1.986712        2.972676
## Hornet Sportabout      Valiant
##           4.425478        5.478024
```

Under a multivariate normal model, a chi-squared quantile can be used as a reference cutoff. This should be interpreted carefully: it is model-based and is not a universal outlier rule.

```
# Reference cutoff motivated by multivariate normal theory
cutoff <- qchisq(.975, df = ncol(X))

# Identify observations beyond the reference cutoff
which(d2 > cutoff)
```

```
## Maserati Bora
##           31
```

```
# Display the corresponding automobile names and distances
data.frame(Car = rownames(X), Mahalanobis2 = d2)[d2 > cutoff, ]
```

```
##           Car Mahalanobis2
## Maserati Bora Maserati Bora      15.86553
```

Ellipsoid Axes Come from Eigenvectors

Quadratic forms also provide the geometry behind multivariate ellipsoids. If

$$\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^T,$$

then the contour

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = c^2$$

can be written in eigenvector coordinates as

$$\sum_{j=1}^p \lambda_j (\mathbf{x}^T \mathbf{e}_j)^2 = c^2.$$

The **eigenvectors** determine the ellipsoid directions, while the **eigenvalues** determine the relative axis lengths.

This is why eigenvalues connect **algebra**, **geometry**, **distance**, and **probability**.

Standardize and Reconstruct Covariance

Square-Root Matrices

For a positive definite matrix

$$\mathbf{A} = \mathbf{P}\mathbf{\Lambda}\mathbf{P}^T,$$

define

$$\mathbf{A}^{1/2} = \mathbf{P}\mathbf{\Lambda}^{1/2}\mathbf{P}^T, \quad \mathbf{A}^{-1/2} = \mathbf{P}\mathbf{\Lambda}^{-1/2}\mathbf{P}^T.$$

These matrices have two complementary interpretations:

$$\mathbf{z} \text{ with covariance } \mathbf{I} \xrightarrow{\mathbf{x} = \boldsymbol{\mu} + \mathbf{A}^{1/2} \mathbf{z}} \mathbf{x} \text{ with covariance } \mathbf{A},$$

which **creates** covariance $\mathbf{A}$, while

$$\mathbf{x} \text{ with covariance } \mathbf{A} \xrightarrow{\mathbf{z} = \mathbf{A}^{-1/2}(\mathbf{x} - \boldsymbol{\mu})} \mathbf{z} \text{ with covariance } \mathbf{I},$$

which **removes** covariance and produces an identity covariance structure.

Before constructing the square root, the spectral decomposition also gives an easy way to reconstruct the ordinary inverse:

$$\mathbf{S}^{-1} = \sum_{j=1}^p \frac{1}{\lambda_j} \mathbf{e}_j \mathbf{e}_j^T.$$

```
# Allocate storage for the spectral components of S^{-1}
temp1 <- array(dim = c(5, 5, 5))

# Reconstruct S^{-1} using sum (1/lambda_i) e_i e_i^T
for (i in 1:5){
  temp1[i, ,] <- (1 / eigen$values[i]) * eigen$vectors[, i] %*% t(eigen$vectors[, i])
}

# Add the components and compare with solve(S)
(out1 <- apply(temp1, 2:3, sum))
```

```
##           [,1]           [,2]           [,3]           [,4]           [,5]
## [1,]  0.1695494031 -0.0006468718  0.0058975274 -0.29977161  0.58997555
## [2,] -0.0006468718  0.0005369064 -0.0003801427  0.02257595 -0.03751089
## [3,]  0.0058975274 -0.0003801427  0.0008208474 -0.02678451  0.02595898
## [4,] -0.2997716134  0.0225759526 -0.0267845083  8.50376340  0.40558365
## [5,]  0.5899755523 -0.0375108878  0.0259589804  0.40558365  7.37641228
```

S_inv

```
##           mpg           disp           hp           drat           wt
## mpg  0.1695494031 -0.0006468718  0.0058975274 -0.29977161  0.58997555
## disp -0.0006468718  0.0005369064 -0.0003801427  0.02257595 -0.03751089
## hp   0.0058975274 -0.0003801427  0.0008208474 -0.02678451  0.02595898
## drat -0.2997716134  0.0225759526 -0.0267845083  8.50376340  0.40558365
## wt   0.5899755523 -0.0375108878  0.0259589804  0.40558365  7.37641228
```

```
# Allocate storage for the spectral components of S^{1/2}
temp2 <- array(dim = c(5, 5, 5))

# Construct S^{1/2} using sum sqrt(lambda_i) e_i e_i^T
for (i in 1:5){
  temp2[i, ,] <- sqrt(eigen$values[i]) * eigen$vectors[, i] %*% t(eigen$vectors[, i])
}

# Add the components to obtain S^{1/2}
out2 <- apply(temp2, 2:3, sum)

# Check S^{1/2} S^{1/2} = S
(out2 %*% out2)
```

```
##           [,1]           [,2]           [,3]           [,4]           [,5]
## [1,]  36.324103 -633.09721 -320.73206  2.1950635 -5.1166847
## [2,] -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## [3,] -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## [4,]  2.195064 -47.06402 -16.45111  0.2858814 -0.3727207
## [5,] -5.116685 107.68420 44.19266 -0.3727207 0.9573790
```

S

```
##           mpg           disp           hp           drat           wt
## mpg  36.324103 -633.09721 -320.73206  2.1950635 -5.1166847
## disp -633.097208 15360.79983 6721.15867 -47.0640192 107.6842040
## hp   -320.732056 6721.15867 4700.86694 -16.4511089 44.1926613
## drat  2.195064 -47.06402 -16.45111  0.2858814 -0.3727207
## wt   -5.116685 107.68420 44.19266 -0.3727207 0.9573790
```

The course slides also use the inverse square root explicitly. We can compute it in the same way without changing the structure of the earlier code.

```
# Construct S^{-1/2} from the same eigenvectors and inverse square-root eigenvalues
S_inv_half <- eigen$vectors %*% diag(1 / sqrt(eigen$values)) %*% t(eigen$vectors)

# Check the whitening identity S^{-1/2} S S^{-1/2} = I
round(S_inv_half %*% S %*% S_inv_half, 8)
```



```
##      [,1] [,2] [,3] [,4] [,5]
## [1,]    1    0    0    0    0
## [2,]    0    1    0    0    0
## [3,]    0    0    1    0    0
## [4,]    0    0    0    1    0
## [5,]    0    0    0    0    1
```

This gives a useful interpretation of Mahalanobis distance. If

$$\mathbf{z} = \mathbf{S}^{-1/2}(\mathbf{x} - \bar{\mathbf{x}}),$$

then

$$\|\mathbf{z}\|^2 = (\mathbf{x} - \bar{\mathbf{x}})^\top \mathbf{S}^{-1}(\mathbf{x} - \bar{\mathbf{x}}).$$

Therefore,

Mahalanobis distance = Euclidean distance after covariance standardization.

Partitioning and Synthesis

Partitioning Random vectors

Sometimes the variables naturally divide into two meaningful groups. We can partition a random vector, its mean, and its covariance matrix conformably:

$$\mathbf{X} = \begin{bmatrix} \mathbf{X}_1 \\ \mathbf{X}_2 \end{bmatrix}, \quad \boldsymbol{\mu} = \begin{bmatrix} \boldsymbol{\mu}_1 \\ \boldsymbol{\mu}_2 \end{bmatrix}, \quad \boldsymbol{\Sigma} = \begin{bmatrix} \boldsymbol{\Sigma}_{11} & \boldsymbol{\Sigma}_{12} \\ \boldsymbol{\Sigma}_{21} & \boldsymbol{\Sigma}_{22} \end{bmatrix}.$$

The diagonal blocks $\boldsymbol{\Sigma}_{11}$ and $\boldsymbol{\Sigma}_{22}$ describe variation and covariation **within** the two groups. The off-diagonal blocks describe association **between** the two groups, with

$$\boldsymbol{\Sigma}_{12} = \boldsymbol{\Sigma}_{21}^\top.$$

Here, we partition the selected automobile variables into two groups:

1. disp, hp, wt,
2. mpg, drat.

```
# Identify the variables in the first block
vars1 <- which(names(mtcars) %in% c("disp", "hp", "wt"))

# Identify the variables in the second block
vars2 <- which(names(mtcars) %in% c("mpg", "drat"))

# Reorder the variables so the two blocks are adjacent
carPar <- mtcars[, c(vars1, vars2)]

# Full covariance matrix after reordering
Sigma <- cov(carPar)

# Within-block covariance for disp, hp, and wt
(Sigma11 <- Sigma[1:3, 1:3])
```

```
##           disp           hp           wt
## disp 15360.7998 6721.15867 107.684204
## hp    6721.1587 4700.86694  44.192661
## wt     107.6842  44.19266   0.957379
```

```
# Within-block covariance for mpg and drat
(Sigma22 <- Sigma[4:5, 4:5])
```

```
##           mpg           drat
## mpg  36.324103 2.1950635
## drat  2.195064 0.2858814
```

```
# Between-block covariance
(Sigma12 <- Sigma[1:3, 4:5])
```

```
##           mpg           drat
## disp -633.097208 -47.0640192
## hp    -320.732056 -16.4511089
## wt     -5.116685  -0.3727207
```

```
# Symmetry implies Sigma21 = Sigma12^T
(Sigma21 <- Sigma[4:5, 1:3])
```

```
##           disp           hp           wt
## mpg  -633.09721 -320.73206 -5.1166847
## drat  -47.06402  -16.45111 -0.3727207
```

```
t(Sigma12)
```

```
##           disp           hp           wt
## mpg  -633.09721 -320.73206 -5.1166847
## drat  -47.06402  -16.45111 -0.3727207
```

If $\mathbf{X}_1$ has dimension $q \times 1$ and $\mathbf{X}_2$ has dimension $(p - q) \times 1$, then

$$\Sigma_{12}$$

must have dimension

$$q \times (p - q).$$

Partitioning is used later for **conditional multivariate normal distributions** and **canonical correlation analysis**.